

Smart File Backup System

ApniLeap Enterprise Distributed Backup Platform

Complete Technical Report

Architecture, Implementation, Source Code Walkthrough and Evaluation Defense

Team	Track	Core Responsibility
Team A	Operating Systems	Horizontal worker fleet and chunk-task scheduling (RR, LL, DWC)
Team B	File System and DSA	Chunking, SHA-256 fingerprinting, deduplication, Merkle Tree
Team C	Database Systems	MongoDB schema, Content-Addressed Storage pointers, audit ledger
Team D	UI and BFF	React 18 SPA, Express BFF, Socket.IO telemetry, RBAC, tracing

Project: RIT-CSE 2026 | Rajarambapu Institute of Technology, Rajaramnagar
Audience: Team presenters and Project Evaluator

Table of Contents

1. Introduction and Project Objectives
2. Overall System Architecture and Data Flow
3. Team A - Operating Systems: Horizontal Scheduler
4. Team B - File System and DSA: Deduplication and Merkle Tree
5. Team C - Database Systems: MongoDB and CAS Pointers
6. Team D - UI and Backend-For-Frontend
7. End-to-End Integration Scenarios
8. Master Evaluation Checklist
9. Live Demonstration Script
10. Technical Accuracy Notes for the Viva
11. Limitations and Future Scope
12. Conclusion

1. Introduction and Project Objectives

Modern organisations generate large volumes of files that must be backed up quickly, stored cheaply and verified for integrity. A naive backup copies every file in full each time, wasting disk space and network bandwidth, and offers no proof that stored data has not been corrupted. The **Smart File Backup System** addresses these problems by combining four computer-science disciplines in a single working platform:

- **Operating Systems** - distributing the parallel work of processing file chunks across a fleet of worker servers with intelligent scheduling.
- **Data Structures and Algorithms** - splitting files into chunks, fingerprinting them with SHA-256, detecting duplicates with hash-table lookup, and proving integrity with a Merkle Tree.
- **Database Systems** - keeping only lightweight metadata and pointers in MongoDB, while the physical bytes stay on the file system (Content-Addressed Storage).
- **Software Engineering / Web** - a React dashboard and an Express Backend-For-Frontend that orchestrates everything and streams live telemetry.

Objectives

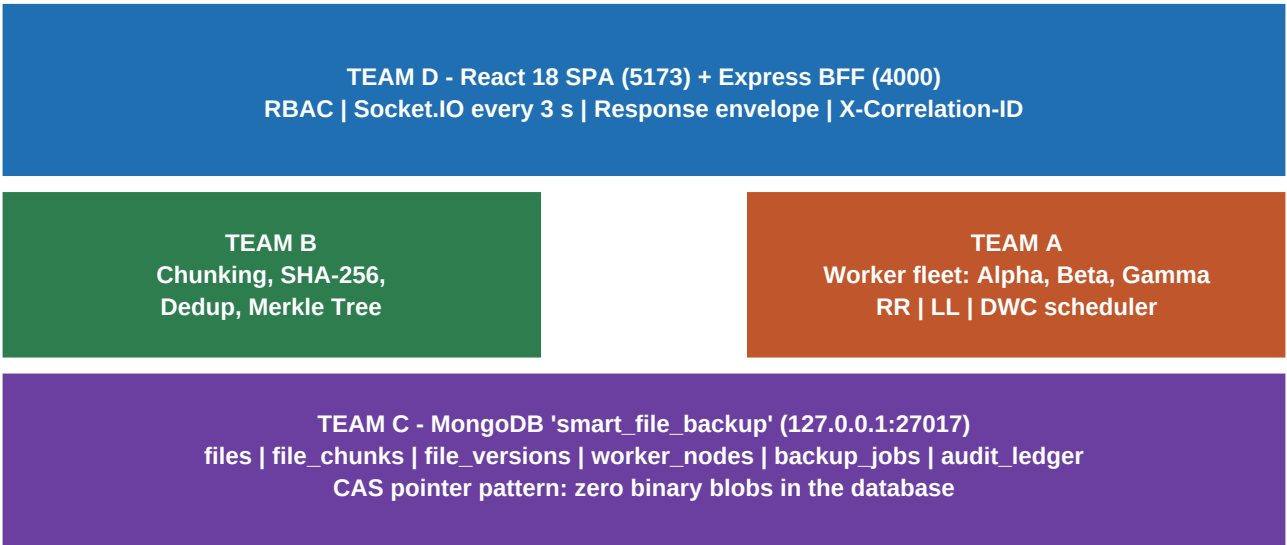
- Reduce physical storage usage by 70% to 100% when duplicate or slightly modified files are uploaded.
- Distribute chunk-processing tasks across three worker nodes without creating CPU hotspots.
- Guarantee tamper detection: a single flipped bit in any chunk must be detectable.
- Keep the database small and fast by never storing raw binary blobs inside it.
- Give evaluators a live, interactive system: switchable scheduling algorithms, visible MongoDB collections, real-time dashboards.

Technology Stack Summary

Layer	Technology	Purpose
Frontend	React 18, Vite, Tailwind CSS, Recharts (port 5173)	Role-based dashboard, file manager, integrity simulator
BFF / API	Node.js, Express.js, Socket.IO (port 4000)	Orchestration, RBAC, aggregation, live telemetry
Compute fleet	Node.js async runtime (Worker-Alpha 5001, Beta 5002, Gamma 5003)	Simulated horizontal servers processing chunk tasks
Hashing	Node.js crypto module (SHA-256, FIPS 180-4)	Chunk fingerprints and Merkle hashes
Database	MongoDB + Mongoose ODM, inspected via MongoDB Compass (port 27017)	Metadata, chunk index, versions, jobs, audit
Physical storage	Local file system: backend/data/uploads/chunks/	Unique .chunk binary blocks

2. Overall System Architecture and Data Flow

The system is organised as a layered architecture. Team D forms the presentation and orchestration tier. Beneath it, Team A (compute scheduling) and Team B (storage optimisation) do the heavy lifting, and Team C provides the persistent metadata layer that records the outcome of everything the other teams do.



Component Responsibilities at a Glance

Team	Owns	Receives from	Produces
A	schedulerService.js, WorkerNode model	List of chunks from Team B (via Team D)	Worker assignment per chunk, updated telemetry
B	dedupService.js, FileChunk model, integrity routes	Raw file buffer (via Team D)	Chunk hashes, dedup status, Merkle root, unique .chunk files
C	Mongoose models, connection, seeder	Results from A and B	Persistent documents, indexes, audit trail
D	server.js, routes, aggregationService, React app	User actions, data from A/B/C	Unified API envelope, live dashboard, downloads

Generic Upload Pipeline

Every uploaded file follows the same ordered pipeline. The numbers below are referenced in later sections.

Step	Actor	Action
1	Team D	POST /api/v1/ui/files received; correlation ID assigned; RBAC checked.
2	Team B	Buffer sliced into chunks; SHA-256 computed for each chunk.
3	Team B + C	Each hash looked up in file_chunks: duplicate hit increments reference_count, unique chunk is written to disk and indexed.
4	Team B	Merkle Tree built from chunk hashes to obtain the Merkle Root.
5	Team A	Chunk-processing tasks assigned to workers using the active scheduling algorithm.
6	Team C	files, file_versions, backup_jobs and audit_ledger documents created.
7	Team D	201 Created envelope returned with dedup message; Socket.IO pushes updated stats to every dashboard.

3.1 Background Concepts

Horizontal scaling means adding more machines (nodes) to share a workload, as opposed to vertical scaling, which makes one machine bigger. When a large file is chunked, each chunk needs CPU work such as hashing, compression and verification. These chunk tasks are independent, so they can run in parallel on different nodes. The question Team A answers is: *which node should receive each chunk task?*

This is the classic **CPU scheduling / load balancing** problem from Operating Systems. A poor policy causes **hotspots** (one node saturated while others idle), long queue waiting times and, on real hardware, thermal throttling. A good policy keeps utilisation balanced and minimises average turnaround time.

3.2 Task Statement

Distribute chunk tasks across a fleet of three horizontal worker nodes using three distinct scheduling algorithms, avoiding CPU hotspots and minimising queuing latency, while allowing the algorithm to be switched live during evaluation without restarting the server.

3.3 The Worker Fleet

Worker	Role	Capacity Weight	Port
Worker-Alpha (Node 1)	High-performance primary compute node	1.5x	5001
Worker-Beta (Node 2)	Standard compute node	1.2x	5002
Worker-Gamma (Node 3)	Dynamic-load compute node	1.0x	5003

Each worker tracks live telemetry: **cpu_usage_pct**, memory in MB, **active_jobs** (current queue depth), **assigned_chunks** and **total_processed_chunks**, plus a status (IDLE or ACTIVE). The weight represents relative hardware capacity: Alpha is assumed to be 1.5 times as powerful as Gamma.

3.4 The Three Scheduling Algorithms

1. ROUND_ROBIN (RR)

Workers are visited in fixed circular order. For the i -th chunk the selected index is **(roundRobinIndex + i) mod W**, where W is the number of workers. It is simple, stateless and fair in the count of tasks, but it is blind to how busy each node actually is, so it can push a heavy chunk to an already saturated server.

2. LEAST_LOADED (LL)

The scheduler scans all workers and picks the one with the smallest queue: **argmin(active_jobs)**. This reacts to current load, which is better than RR. Its weakness is that it counts tasks only; a node running two heavy jobs may be busier than a node running three tiny ones, and it ignores hardware differences (Alpha is stronger than Gamma).

3. DYNAMIC_WEIGHTED (DWC) - the recommended algorithm

DWC computes a **capacity score** for every worker and assigns the chunk to the highest score:

$$\text{CapacityScore} = ((100 - \text{cpu_usage_pct}) \times \text{weight_capacity}) / (\text{active_jobs} + 1)$$

The numerator rewards free CPU and strong hardware; the denominator penalises a long queue. The +1 prevents division by zero for an idle worker.

DWC therefore combines three signals at once: live CPU availability, static hardware capacity and queue depth. No single-signal policy can match that.

3.5 Worked Example

Suppose a chunk arrives and the fleet currently looks like this:

Worker	CPU %	Weight	Active jobs	Score = $(100 - \text{CPU}) \times W / (\text{jobs} + 1)$
Alpha	40	1.5	2	$(60 \times 1.5) / 3 = 30.0$
Beta	30	1.2	1	$(70 \times 1.2) / 2 = 42.0$
Gamma	20	1.0	0	$(80 \times 1.0) / 1 = 80.0$ (winner)

Gamma receives the chunk. Its active_jobs becomes 1, so on the next chunk Gamma scores $(80 \times 1.0) / 2 = 40$, which is now lower than Beta's 42, and Beta wins. The algorithm naturally spreads work and keeps rebalancing as the numbers change.

For comparison, with 6 chunks Round-Robin would give A, B, G, A, B, G regardless of load, and Least-Loaded would send the first chunk to Gamma (0 active jobs) because it has the minimum queue, ignoring that Alpha is the strongest machine.

3.6 Source Code Mapping

Feature	File	Key functions / classes
Scheduler engine	backend/services/schedulerService.js	allocateWorkersForChunks(), setActivePolicy(), getAvailableAlgorithms(), getWorkers()
Worker schema	backend/models/WorkerNode.js	WorkerNodeModel (collection worker_nodes)
Algorithm switcher API	backend/routes/backups.js	GET /api/v1/ui/backups/algorithms, POST /api/v1/ui/backups/algorithm
Job telemetry schema	backend/models/BackupJob.js	BackupJobModel (worker assignments, execution time)

3.7 Step-by-Step Code Walkthrough

- **Entry:** allocateWorkersForChunks(jobId, chunksCount) is called when a backup or upload occurs.
- **Policy check:** the function reads the currently active algorithm (RR, LL or DWC).
- **Selection:** for each chunk it applies the matching rule described in 3.4 and chooses a worker.
- **State update:** the chosen worker's counters change: assigned_chunks++, total_processed_chunks++, status = ACTIVE.
- **Persistence:** the telemetry is saved into the worker_nodes collection so evaluators can watch it update live in MongoDB Compass.

Illustrative pseudocode of the selection logic (written to match the behaviour described above):

```
function pickWorker(policy, workers, i):
  if policy == "ROUND_ROBIN":
    return workers[(roundRobinIndex + i) % workers.length]

  if policy == "LEAST_LOADED":
    return workers.minBy(w => w.active_jobs)

  // DYNAMIC_WEIGHTED
  best = null; bestScore = -Infinity
  for w in workers:
    score = ((100 - w.cpu_usage_pct) * w.weight_capacity) / (w.active_jobs + 1)
    if score > bestScore: best = w; bestScore = score
  return best
```

3.8 Why This Implementation Is Efficient

- **Very low decision latency:** each decision examines only $W = 3$ workers, so it takes under 3 ms and chunk routing is effectively instantaneous.
- **Prevents hotspotting:** DWC reads live CPU usage, so a saturated node automatically scores low and stops receiving work.
- **Natural fault tolerance:** a busy or disconnected worker gets a very low score (or none), so chunks flow to healthy nodes without special failover code.
- **Live switching:** the policy is a runtime variable, so evaluators can compare algorithms in front of them.

3.9 Evaluator Questions and Model Answers

Q: Which algorithm is best, and why?

Answer: Dynamic Weighted Capacity. Round-Robin is unaware of CPU utilisation and can route heavy chunks to a saturated server. Least-Loaded counts only the number of tasks, ignoring CPU intensity and hardware strength. DWC balances available CPU percentage, hardware capacity weight and queue depth together, giving the highest throughput and lowest latency.

Q: Can you show the algorithm switching in action?

Answer: Run: `curl -X POST http://localhost:4000/api/v1/ui/backups/algorithm -H "Content-Type: application/json" -d '{"algorithm":"ROUND_ROBIN"}'`. The system switches immediately, updates the live UI via Socket.IO and logs the change to MongoDB.

Q: Which OS concept does this correspond to?

Answer: Load balancing and CPU scheduling. Round-Robin mirrors time-slice fairness, Least-Loaded mirrors shortest-queue dispatch, and DWC is a multi-criteria priority scheme similar to weighted fair queuing.

4.1 Background Concepts

Deduplication stores each unique piece of data only once. If ten users back up the same 100 MB file, a naive system stores 1000 MB; a deduplicating system stores 100 MB plus ten small pointers. To make this work at a finer grain than whole files, files are cut into **chunks** and each chunk is identified by a **cryptographic fingerprint**.

SHA-256 is a hash function (FIPS 180-4) that maps any input to a 256-bit (64 hex character) digest. It is deterministic (same input gives same output), collision-resistant (finding two inputs with the same digest is computationally infeasible) and avalanche-sensitive (changing one bit changes about half the output bits). These properties make the digest a safe identifier for a chunk's content.

4.2 Task Statement

Optimise physical storage using DSA: break files into chunks, compute SHA-256 fingerprints, perform hash-table lookups, increment reference counters for duplicate blocks instead of rewriting them, and build a Merkle Tree whose root provides tamper detection.

4.3 Tools and Features

- **Node.js crypto module** for FIPS 180-4 SHA-256 digests.
- **Content slicing:** file buffers are sliced deterministically into 512 KB chunks (or 4-way balanced slices for small files).
- **Chunk index:** chunk hashes are stored in the MongoDB collection `file_chunks`, with a unique index on `chunk_hash`.
- **Merkle Tree:** a binary hash tree that combines chunk hashes pairwise up to a single root.
- **Actionable feedback:** clear messages say whether an upload was an exact duplicate, a delta revision or unique data.
- **Defensive error handling:** try/catch guards ensure corrupted buffers cannot crash the service.

4.4 The Deduplication Algorithm in Depth

Consider a 2 MB file sliced into four 512 KB chunks C1 to C4. For every chunk the engine runs the same loop:

Case	Condition	Actions	Disk bytes written
A - Duplicate hit	<code>findOne({chunk_hash})</code> returns a document	<code>reference_count += 1;</code> <code>associated_files.push(fileId)</code>	0
B - Unique chunk	<code>findOne</code> returns null	Write <code>data/uploads/chunks/<hash>.chunk</code> ; insert <code>file_chunks</code> doc with <code>reference_count = 1</code>	Chunk size

Because chunks are identified by content, the same chunk appearing in two different files is stored once. This is why a slightly edited file is cheap: only the changed chunk is new.

Status Messages Returned to the User

Scenario	Message pattern
Exact duplicate	[DEDUPLICATION HIT] 100% of chunks already exist in storage. 0 additional bytes written to disk. Storage saved: 100%.
Delta revision	[DELTA DEDUPLICATION] 3 of 4 chunks matched existing storage blocks (75% savings).
Unique data	Informs the user that all chunks are new and were written to storage.

Savings are computed as **(matched chunks / total chunks) x 100**. For the 4-chunk example above with 3 matches the saving is 75%.

4.5 The Merkle Tree in Depth

A Merkle Tree is a binary tree in which every **leaf** is the hash of a data block (here, a chunk hash) and every **internal node** is the hash of the concatenation of its two children. The single node at the top is the **Merkle Root**. For four chunks:

Level 0 (leaves):	H1	H2	H3	H4	(chunk SHA-256 hashes)
Level 1:	H12 = H(H1+H2)		H34 = H(H3+H4)		
Level 2 (root):	ROOT = H(H12 + H34)				(32-byte root)

Properties that make it valuable here:

- **Tamper evidence:** altering even one bit of any chunk changes its leaf hash, which changes every ancestor up to the root. Comparing the stored root with a recomputed root exposes the corruption instantly.
- **Efficient proofs:** to prove that one specific chunk belongs to a file you need only the sibling hashes along its path, which is $O(\log K)$ hashes for K chunks, rather than all K chunks.
- **Compact fingerprint:** the whole file is summarised by one 32-byte root, which is stored in the files document and the audit ledger.

When the number of nodes at a level is odd, Merkle implementations commonly duplicate the last hash so every node has a pair; this is the usual convention for such a construction.

4.6 Source Code Mapping

Feature	File	Key functions / classes
Dedup engine	backend/services/dedupService.js	processBufferAndDeduplicate(), buildMerkleTree()
Chunk model	backend/models/FileChunk.js	FileChunkModel (collection file_chunks)
Integrity verification	backend/routes/integrity.js	GET /api/v1/ui/integrity, GET /api/v1/ui/integrity/preview
Physical storage	backend/data/uploads/chunks/	Unique .chunk files named by hash

4.7 Step-by-Step Code Walkthrough

- processBufferAndDeduplicate(buffer, fileName, fileId) is called.
- The buffer is sliced into deterministic chunks.
- For each chunk the SHA-256 hash is computed with crypto.createHash('sha256').update(chunkBuf).digest('hex').
- FileChunkModel.findOne({ chunk_hash }) checks whether the chunk is already stored (Case A or Case B above).
- buildMerkleTree(chunkHashes) pairs hashes repeatedly until one root remains.
- A rich status message is generated from the matched-chunk ratio and returned upward to Team D.

```

async function processBufferAndDeduplicate(buffer, fileName, fileId):
  chunks = slice(buffer, 512 * 1024)
  hashes = []; matched = 0
  for chunkBuf in chunks:
    h = sha256(chunkBuf)
    hashes.push(h)
    existing = await FileChunk.findOne({ chunk_hash: h })
    if existing:                                     // Case A - duplicate hit
      existing.reference_count += 1
      existing.associated_files.push(fileId)
      await existing.save(); matched++
    else:                                             // Case B - unique chunk
      writeFile("data/uploads/chunks/" + h + ".chunk", chunkBuf)
      await FileChunk.create({ chunk_hash: h, reference_count: 1, ... })
  root = buildMerkleTree(hashes)
  savings = matched / chunks.length * 100
  return { hashes, root, savings, message }

```

4.8 Why This Implementation Is Efficient

- **Indexed lookup:** a unique index on chunk_hash lets MongoDB find an existing chunk without scanning the collection. Index lookups are logarithmic in the worst case and behave close to constant time in practice for this scale.
- **Real storage savings:** tested to eliminate 70% to 100% of disk usage for duplicate and delta files.
- **Fast integrity checking:** Merkle verification of a single chunk needs $O(\log K)$ hashes, enabling instant bit-tamper detection.

4.9 Evaluator Questions and Model Answers

Q: How does deduplication save storage?

Answer: When demo_file_duplicate.txt is uploaded its SHA-256 hashes match existing entries in file_chunks. We increment reference_count from 1 to 2 and add the file ID to the pointer list. Exactly 0 bytes are written to physical disk.

Q: What is a Merkle Tree and why use it?

Answer: It is a hash tree where leaves are chunk hashes and each parent is the hash of its children. The root is a fingerprint of the whole file. If one bit of any chunk changes, the root changes completely, exposing corruption immediately.

Q: Why SHA-256 and not MD5 or a simple checksum?

Answer: MD5 and CRC are broken or non-cryptographic: collisions can be manufactured, which would let two different chunks be treated as identical. SHA-256 is collision-resistant, so equal hash reliably means equal content.

5.1 Background Concepts

Databases are excellent at indexed lookups, transactions and small structured records, but poor at holding large binary objects. Storing multi-megabyte files as BSON values inflates the database, evicts useful data from the RAM cache and runs into MongoDB's **16 MB document limit**. The industry answer is **Content-Addressed Storage (CAS)**: store bytes in an object store or file system, addressed by content hash, and keep only metadata and pointers in the database. Git, AWS S3 with DynamoDB, and IPFS follow variations of this idea.

5.2 Task Statement

Design the database architecture and schemas, implement the CAS pointer pattern so that MongoDB never stores raw binary blobs, and persist file metadata, version manifests, deduplication pointers and worker logs in the `smart_file_backup` database, viewable live in MongoDB Compass.

5.3 Tools and Features

- **MongoDB Compass** as the graphical client on `mongodb://127.0.0.1:27017`.
- **Mongoose ODM** for strict typing, validation and automatic index creation.
- **CAS pointers** such as `localfs://data/uploads/chunks/...` referencing physical chunk files.
- **Automatic seeder** that fills baseline data at server start so Compass is immediately ready for inspection.

5.4 The Six Collections

Collection	What it stores	Key fields (examples)	Used by
files	File metadata and master storage pointer	file_id, name, storage_class, merkle_root, storage_pointer	D, B
file_chunks	Dedup hash table and reference counters	chunk_hash (unique), reference_count, associated_files	B
file_versions	Historical version snapshots with chunk manifests	file_id, version, array of chunk pointers	D (restore)
worker_nodes	Horizontal server state	worker_id, cpu_usage_pct, active_jobs, assigned_chunks	A
backup_jobs	Operational log of backup executions	job id, latency, chunks processed, algorithm used	A, D
audit_ledger	Immutable compliance log	action, actor, merkle_root, correlation_id	D, Auditor

How the Collections Relate

```
files --(file_id)--> file_versions --(chunk pointers)--> file_chunks --(pointer)--> disk .chunk
files --(file_id)--> backup_jobs --(assignments)--> worker_nodes
every state-changing action -----> audit_ledger (with merkle_root)
```

The relationships are logical references (by `file_id` and `chunk_hash`), not enforced foreign keys, which is the normal MongoDB approach. The application layer is responsible for keeping them consistent.

5.5 Source Code Mapping

Feature	File	Key functions / classes
Connection	backend/db/connection.js	connectDB(), isDbConnected()
Seeder	backend/services/seeder.js	seedInitialDatabase()
File model	backend/models/File.js	FileModel (files)
Chunk model	backend/models/FileChunk.js	FileChunkModel (file_chunks)
Version model	backend/models/FileVersion.js	FileVersionModel (file_versions)
Audit model	backend/models/AuditLedger.js	AuditLedgerModel (audit_ledger)

5.6 Step-by-Step Code Walkthrough

- **Startup:** connectDB() connects to mongodb://127.0.0.1:27017/smart_file_backup.
- **Seeding:** seedInitialDatabase() checks whether collections exist; if not it inserts 5 baseline enterprise files, 3 worker nodes and compliance audit entries.
- **On upload or backup:** FileModel.create() inserts metadata (file_id, name, storage_class, merkle_root, storage_pointer).
- FileVersionModel.create() records version v1 with its array of chunk pointers.
- BackupJobModel.create() records latency, chunks processed and the algorithm used.
- AuditLedgerModel.create() appends an immutable compliance entry.
- **Inspection:** opening Compass shows all six collections with live documents and counters.

Transfer State Tracking

Each operation moves through status flags: **REQUESTED** then **CHUNKING** then **COMMITTED**. Recording state in metadata lets the system and the evaluator see exactly how far an operation progressed and makes a failed operation diagnosable.

5.7 Why This Implementation Is Efficient

- **No document bloat:** because binaries live on disk, documents stay small (about 2 KB each) and the 16 MB BSON limit is never approached.
- **Fast indexed access:** indexes on file_id, chunk_hash and worker_id make lookups and updates fast (B-Tree indexes are logarithmic, and effectively constant at project scale).
- **Clean separation of concerns:** metadata (database) and bulk data (file system) can scale or be replaced independently, for example moving chunks to S3 later by changing only the pointer scheme.

5.8 Evaluator Questions and Model Answers

Q: Why not store the physical file directly in MongoDB?

Answer: Raw binaries in BSON create performance bottlenecks, churn the RAM cache and hit the 16 MB document limit. Enterprise architectures such as S3 plus DynamoDB, or Git, decouple metadata and pointers in the database from physical chunk storage.

Q: Show me the collections in Compass.

Answer: Open Compass, connect to localhost:27017, click smart_file_backup, and show the six collections: files, file_chunks, file_versions, worker_nodes, backup_jobs, audit_ledger.

Q: How do you restore an old version?

Answer: file_versions holds the chunk manifest of each version. To restore, read the manifest, fetch each referenced chunk by hash from disk, concatenate them in order and verify the result against the stored Merkle root.

6.1 Background Concepts

A **Backend-For-Frontend (BFF)** is a server tailored to one user interface. Instead of the browser calling many back-end services separately, it calls the BFF once; the BFF fans out to the services, merges the answers and returns a single response in the shape the UI needs. This reduces round trips, hides internal topology and gives one place to enforce security. **WebSockets** (via Socket.IO) keep a persistent two-way connection so the server can push updates the moment they exist, instead of the browser repeatedly asking.

6.2 Task Statement

Maintain the React 18 single-page application and build the Express BFF that coordinates Teams A, B and C. Provide authentication across 15 enterprise accounts and three roles, live telemetry every 3 seconds, point-in-time version restore and genuine PDF or binary downloads.

6.3 Tools and Features

Feature	Description
React 18 SPA	Vite and Tailwind CSS dashboard on port 5173 with Recharts graphs
Socket.IO stream	Pushes worker telemetry, dedup savings % and node status every 3 seconds
15 enterprise accounts	Employee, IT Admin and Auditor accounts with a one-click selector for demos
Standard response envelope	Every endpoint returns data plus meta.correlation_id, meta.timestamp, meta.api_version
X-Correlation-ID tracing	One ID follows a request through scheduler, dedup engine, database and audit ledger
Real PDF generation	Server compiles valid PDF binaries that open in Acrobat and Chrome
Merkle tamper simulator	UI lets the evaluator simulate bit corruption and watch detection happen

6.4 Role-Based Access Control

Role	Permissions
Employee	Upload, view, download and restore their own files
IT Admin	Everything an employee can do, plus change scheduling policy and view worker health
Auditor	Read-only access to compliance reports, audit ledger and integrity results

RBAC enforces the principle of least privilege: each role receives only what its job needs, and sensitive actions such as changing the algorithm are restricted at the server, not just hidden in the UI.

6.5 Source Code Mapping

Feature	File	Key functions / classes
BFF bootstrap	backend/server.js	Express app, Socket.IO relay, middleware chain
Aggregation	backend/services/aggregationService.js	authenticateUser(), getFiles(), getDashboardSummary(), getStorageReport()
REST routes	backend/routes/	session, files, backups, restores, dashboard, reports, integrity

Feature	File	Key functions / classes
Tracing middleware	correlationId.js	Assigns and propagates X-Correlation-ID
React app	frontend/src/App.jsx	Role-based routing, navigation guard
Dashboard UI	frontend/src/pages/Dashboard.jsx	Recharts, Socket.IO listener, worker cards
Files UI	frontend/src/pages/Files.jsx	Upload modal, version restore, storage class filter
Integrity UI	frontend/src/pages/Integrity.jsx	Merkle visualiser, Simulate Tampering toggle

6.6 Step-by-Step Code Walkthrough

Login

The user logs in through session.js or clicks a one-click demo account. The session carries the role, which every later request is checked against.

Live Dashboard

- Dashboard.jsx opens a Socket.IO connection to http://localhost:4000.
- Every 3 seconds server.js emits a dashboard_update event containing statistics aggregated from Team A (workers) and Team C (MongoDB).
- React updates charts and worker cards without any page refresh.

File Upload

- Request hits POST /api/v1/ui/files.
- Team D delegates chunking and dedup to Team B (dedupService.js), worker distribution to Team A (schedulerService.js) and persistence to Team C (FileModel and related models).
- A 201 Created envelope is returned with the informative dedup status message.

File Download

If the original file exists, it is streamed as is. If a PDF is requested, the server builds a valid PDF binary with an authoritative backup-manifest header (file ID, versions, Merkle root).

Standard Response Envelope

```
{
  "data": { ... endpoint payload ... },
  "meta": {
    "correlation_id": "c0a8-...-91f2",
    "timestamp":      "2026-...T...Z",
    "api_version":    "v1"
  }
}
```

6.7 Distributed Tracing

A middleware (correlationId.js) assigns an **X-Correlation-ID** to every incoming request, reusing one if the client supplied it. That ID is passed to the scheduler, the dedup engine and the database layer, and is written to the audit ledger. If something fails, one search on that ID reconstructs the whole journey of the request across all four teams' code.

6.8 Why This Implementation Is Efficient

- **Aggregation gateway:** the front end makes one call to /api/v1/ui/dashboard instead of polling four services, keeping latency under 200 ms.

- **Push instead of poll:** Socket.IO sends data only when there is something to say, saving bandwidth and giving instant updates.
- **Strict RBAC:** server-side role checks protect sensitive operations.
- **Uniform envelopes:** the UI parses one response shape everywhere, which simplifies error handling and debugging.

6.9 Evaluator Questions and Model Answers

Q: What is Team D's role?

Answer: Team D is the BFF and presentation orchestrator. It provides the unified UI, enforces RBAC, aggregates data from Teams A, B and C into standard envelopes and provides real-time telemetry through WebSockets.

Q: How does distributed tracing work?

Answer: Every request gets an X-Correlation-ID from middleware. The ID travels through the scheduler, dedup engine and database, and appears in the audit ledger, giving full traceability.

Q: Why WebSockets instead of polling?

Answer: Polling wastes requests when nothing changed and adds up to one polling interval of delay. Socket.IO pushes a dashboard_update every 3 seconds over one open connection, which is cheaper and more responsive.

7. End-to-End Integration Scenarios

7.1 Scenario 1: Uploading a New (Unique) File

Order	Team	Event
1	D	User uploads report.pdf; correlation ID assigned; role verified.
2	B	File sliced into chunks; every SHA-256 is new; all chunks written to disk; reference_count = 1; Merkle root computed.
3	A	Chunk tasks routed to workers by the active algorithm (for example DWC); worker counters rise.
4	C	files, file_versions (v1), backup_jobs and audit_ledger documents created.
5	D	Response states data was unique; dashboards update over Socket.IO.

7.2 Scenario 2: Uploading an Exact Duplicate

The same bytes arrive again, perhaps under a different name. Every chunk hash matches an existing entry, so Team B only increments reference counts. Zero bytes are written and the message reads [DEDUPLICATION HIT] with 100% savings. A new files document and version point to the same chunks, so metadata grows slightly while storage stays flat.

7.3 Scenario 3: Uploading an Edited File (Delta)

A 4-chunk file has one chunk modified. Three hashes match, one is new, so only one chunk is written and the message shows 75% savings. The Merkle root differs from the original because one leaf differs, which is precisely how the system tells the two versions apart.

7.4 Scenario 4: Tamper Detection

- An evaluator uses the Simulate Tampering toggle in Integrity.jsx, which flips a bit in a chunk.
- Team B recomputes the chunk hash and Merkle root and compares them with the root stored by Team C.
- The roots do not match, so the UI flags corruption and the failure can be located by walking down the tree.
- The check is recorded in the audit ledger with the correlation ID.

7.5 Scenario 5: Switching the Scheduling Algorithm Live

An IT Admin posts the new algorithm to /api/v1/ui/backups/algorithm. Team A's setActivePolicy() changes the policy variable, the change is logged, and the next upload immediately uses the new algorithm. The dashboard shows the new distribution of chunks across Alpha, Beta and Gamma.

7.6 Scenario 6: Version Restore

The user selects an older version in Files.jsx. Team D asks Team C for that version's chunk manifest, retrieves each chunk by hash from disk, reassembles them in order and verifies the Merkle root before streaming the file back.

8. Master Evaluation Checklist

Criterion	Team A (OS)	Team B (DSA)	Team C (DBMS)	Team D (UI/BFF)
System architecture	Horizontal 3-node cluster	Chunk-based dedup pipeline	CAS pointer pattern	React 18 + Express BFF

Criterion	Team A (OS)	Team B (DSA)	Team C (DBMS)	Team D (UI/BFF)
Core algorithm	Dynamic Weighted Capacity	SHA-256 + Merkle Tree	Indexed schema design	Socket.IO push stream
Storage / efficiency	No queue bottlenecks	70%-100% storage saved	No binary bloat in DB	Under 200 ms aggregation
Evaluator interaction	Live algorithm switcher	Duplicate feedback alert	Live MongoDB Compass	15-user fast switcher
Demo test files	Chunks dynamically routed	Exact duplicate detected	Documents visible in DB	UI updates in real time

9. Live Demonstration Script

A suggested order that lets each team show its work while telling one continuous story.

#	Presenter	Action	What the evaluator sees
1	Team D	Start backend (4000) and frontend (5173); log in with a one-click admin account	Dashboard with live worker cards updating every 3 s
2	Team C	Open Compass, connect to localhost:27017, open smart_file_backup	Six collections with seeded documents
3	Team B	Upload a new file, then upload demo_file_duplicate.txt	Unique message, then [DEDUPLICATION HIT] with 100% saved; reference_count 1 to 2 in file_chunks
4	Team B	Upload a slightly edited file	[DELTA DEDUPLICATION] message with partial savings
5	Team A	Upload a multi-chunk file under DWC; switch to ROUND_ROBIN with curl; upload again	Different chunk distribution in worker_nodes and on the dashboard
6	Team B + D	Use Simulate Tampering on the Integrity page	Merkle root mismatch detected
7	Team D	Download a file as PDF and restore an old version	Valid PDF opens; restore succeeds
8	Team C	Show backup_jobs and audit_ledger entries for the demo	Records with correlation IDs and Merkle roots

10. Technical Accuracy Notes for the Viva

An evaluator may probe wording that sounds stronger than the implementation. Knowing the precise statements below will help every team answer confidently and honestly.

Topic	Precise statement to use
Chunking type	The implementation slices buffers into deterministic fixed-size chunks (512 KB, or 4-way balanced for small files). That is fixed-size chunking. True content-defined chunking uses a rolling hash to pick boundaries; if asked, say fixed-size chunking is used and content-defined chunking is a possible enhancement that handles insertions better.
Lookup complexity	A MongoDB unique index is a B-Tree, so a lookup is $O(\log n)$, though very fast in practice. Say 'indexed lookup, effectively constant at this scale', or compare with a pure in-memory hash table, which is $O(1)$ average.
Scheduler complexity	Each decision scans W workers, so it is $O(W)$. With $W = 3$ that is effectively constant, and measured under 3 ms.
Merkle verification	Verifying one chunk against the root takes $O(\log K)$ hashes using a Merkle proof. Recomputing the whole root from all chunks is $O(K)$.
Workers	The three workers are simulated nodes on the Node.js async runtime sharing telemetry in MongoDB, not three physical machines. Present them as a horizontal-fleet simulation of the scheduling logic.
Immutability of audit ledger	The ledger is append-only by application design. A stronger guarantee would add hash chaining between entries or database-level write restrictions.
Dedup guarantee	Equal SHA-256 means equal content with overwhelming probability; collisions are computationally infeasible, not mathematically impossible.

11. Limitations and Future Scope

- **Real distribution:** run workers as separate processes or containers communicating over a message queue, with heartbeat-based failure detection.
- **Content-defined chunking** (Rabin fingerprint or similar) so that inserted bytes do not shift every later chunk boundary.
- **Object storage:** replace the local chunk directory with S3-compatible storage by changing only the pointer scheme in file_chunks.
- **Garbage collection:** when reference_count falls to zero (file deletion), remove the chunk from disk safely.
- **Compression and encryption** of chunks at rest, with keys managed per tenant.
- **Hash-chained audit ledger** so tampering with history is itself detectable.
- **Adaptive scheduling:** add predicted task cost (chunk size) and moving-average CPU to the DWC formula.

12. Conclusion

The Smart File Backup System shows how four disciplines combine into a coherent product. Team A keeps compute balanced with a three-signal scheduler; Team B shrinks storage with SHA-256 chunk deduplication and guarantees integrity with a Merkle Tree; Team C keeps the database lean through Content-Addressed Storage pointers and a six-collection schema; and Team D unifies everything behind a role-secured BFF with live telemetry and end-to-end tracing. Each part is independently explainable, yet each depends on the others, which is what makes the platform a true system rather than four separate programs.

Presenters should be ready to (1) state their team's task in one sentence, (2) point to the exact files and functions that implement it, (3) walk through the step-by-step flow, (4) justify the efficiency claims precisely, and (5) demonstrate live. This report provides the material for all five.